

TECHNICAL SPECIFICATION DOCUMENT
EMVCO STANDARD QR GENERATOR
UTILITY REQUIREMENT



JazzCash

TABLE OF CONTENTS

TABLE OF CONTENTS	2
1 DOCUMENT INFORMATION	3
2 OVERVIEW	4
3 FEATURES	4
4 ARCHITECTURE	4
5 INPUT & OUTPUT	5
6 DLL FILE LINK AND HOW TO USE IN YOUR PROJECT	7
7 INTEGRATION IN WEB API	8
8 DEPENDENCIES	8
9 NOTES & LIMITATIONS	9

1 DOCUMENT INFORMATION

Category	Information
Organization	Jazz Cash
Document	Technical Specification Document
Filename	EMVCo Standard QR Generator Utility Requirement
Author	Muhammad Atif Siddiq
Reviewer	Khurum Abdullah
Subject	EMVCo Standard QR Generator Utility Requirement
Version	1.0
Control status	Confidential
Distribution	Software
Disclaimer	All rights reserved by Jazz Cash

2 OVERVIEW

The EMV QR Utility is a .NET-based library designed to generate EMV-compliant QR Codes. It converts merchant and transaction information into a valid EMVCo QR string, computes CRC checksum, and provides QR images in both PNG and SVG formats.

This utility can be distributed as a DLL and consumed in any .NET project (Framework ≥ 4.6 or .NET Core ≥ 3.1).

3 FEATURES

- Generate EMV QR string according to EMVCo specifications
- Automatic CRC16-CCITT checksum calculation
- Output formats:
 - QR String
 - Base64 PNG
 - Base64 SVG
- Support for dynamic tags via nested JSON / Dictionary
- Lightweight and dependency on QRCode library

4 ARCHITECTURE

- **Main Class**
EmvQrUtility
- **Core Methods**

Method	Description	Return Type
Generate(Dictionary<string, object> tags)	Builds EMV QR string, adds CRC, and generates QR images.	Tuple<string, string, string>
BuildEmvString(Dictionary<string, object> tags)	Recursively builds EMV string from key-value pairs.	string
CalculateCrc16(string input)	Calculates CRC16 checksum for EMV QR compliance.	string
GeneratePng(string text)	Creates Base64 PNG QR Code.	string
GenerateSvg(string text)	Creates Base64 SVG QR Code.	string

5 INPUT & OUTPUT

Example Input (JSON Request)

```
{
  "tags": {
    "00": "01",
    "01": "11",
    "26": {
      "00": "05"
    },
    "28": {
      "00": "8af32dd79b2247479b6fc2f793173441",
      "01": "WMBLPKKA",
      "02": "PK72JCMA2902923071557873"
    },
    "52": "5411",
    "53": "586",
    "54": "1",
    "58": "PK",
    "59": "Saad Zikria Merchant",
    "60": "Islamabad",
    "62": {
      "01": "100003051828",
      "02": "+92-3071557873",
      "03": "Saad Zikria Merchant",
      "06": "03071557873",
      "07": "980297198",
      "08": "RAAST Bill Payment"
    }
  },
  "isImageRequired": true
}
```

Example Output

```
{
  "qrString": "00020101021126...6304A1B2",
  "base64Png": "iVBORw0KGgoAAAANSUhEUgAA...",
  "base64Svg": "PHN2ZyB2ZXJzaW9uPSIxLjEi..."
}
```

Details of the Data Objects in the Standard QR

EMV Co Tag ID	Name	Format N-Number ans- Alphanumeric Special S-String	Length (In characters)
00	Payload Format Indicator (The value “01” is fixed and must not be modified)	N	02
01	Point of Initiation Method (The value “11” is fixed and must not be modified)	N	02
26	Merchant Account Information	ans	Var upto 99
26-00	Unique Identifier for JazzCash (The value “05” is fixed and must not be modified).	ans	Var upto 02
28	Additional Data Field Template (used By Raast)	ans	Var upto 99
28-00	Globally Unique Identifier (UUID) without dashes “-“	ans	Var upto 32
28-01	BIC Code (The value “WMBLPKKA” is fixed and must not be modified)	ans	Var upto 08
28-02	IBAN (Shared with Partner when the merchant account is created on CPS)	ans	Var upto 24
52	Merchant Category Code	N	04
53	Transaction Currency	N	03
54	Transaction Amount	ans	Var upto 13
58	Country Code	ans	02
59	Merchant Name	ans	Var upto 25
60	Merchant City	ans	Var upto 15
62	Additional Data Field Template	S	Var upto 99
62-01	Bill Number	ans	Variable up to 25
62-02	Mobile Number	ans	Variable up to 15
62-03	Store Label	ans	Variable up to 25
62-06	Customer Label	ans	Variable up to 25
62-07	Terminal Label	ans	Variable up to 25
62-08	Purpose of Transaction (The value “RAAST Bill Payment” is fixed and must not be modified)	ans	Variable up to 25

6 DLL FILE LINK AND HOW TO USE IN YOUR PROJECT

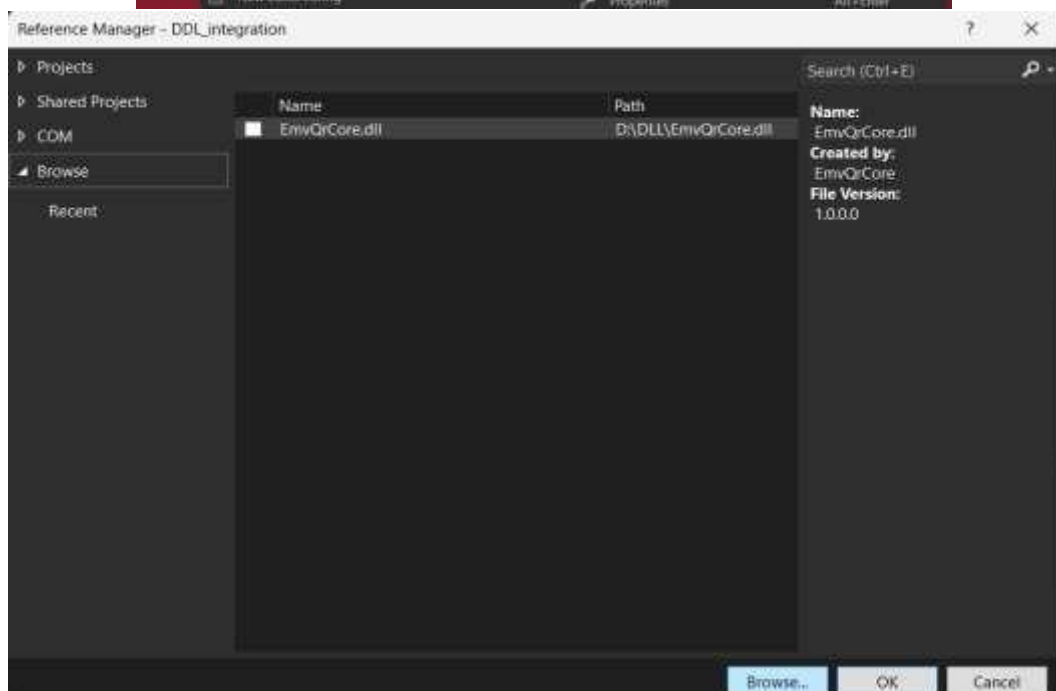
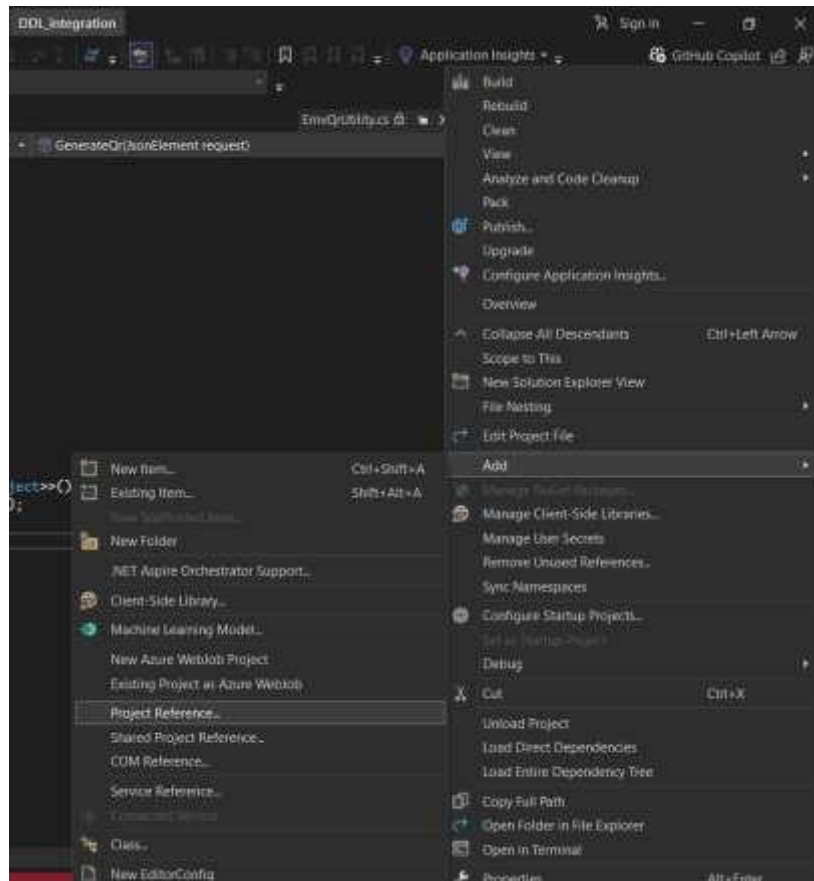
DLL FILE LINK:

Please find below the link for DLL Files.

<https://we.tl/t-R5lNZplhZs>

Step 1: Reference the DLL

- Copy EmvQrNet.dll into your project
- Add reference via Visual Studio → References → Add Reference → Browse → Select DLL



Step 2: Import Namespaces

```
using EmvQrNet;  
using System.Collections.Generic;
```

Step 3: Call the Utility

```
var tags = new Dictionary<string, object>  
{  
    { "00", "01" },  
    { "01", "12" },  
    { "52", "0000" },  
    { "53", "586" },  
    { "54", "100.00" },  
    { "58", "PK" },  
    { "59", "Test Merchant" },  
    { "60", "Karachi" }  
};  
var result = EmvQrUtility.Generate(tags);  
// Access results  
Console.WriteLine("QR String: " + result.Item1);  
Console.WriteLine("Base64 PNG: " + result.Item2);  
Console.WriteLine("Base64 SVG: " + result.Item3);
```

7 INTEGRATION IN WEB API

If consuming in an API:

```
[HttpPost]  
[Route("generate")]  
public IActionResult GenerateQr([FromBody] Dictionary<string, object> tags)  
{  
    var result = EmvQrUtility.Generate(tags);  
    return Ok(new  
    {  
        qrString = result.Item1,  
        base64Png = result.Item2,  
        base64Svg = result.Item3  
    });  
}
```

8 DEPENDENCIES

- .NET Framework 4.6+ or .NET Core 3.1+
- QRCode v1.6.0 (NuGet package)

9 NOTES & LIMITATIONS

- Ensure all EMV tags follow EMVCo specification
- CRC16 checksum is automatically appended (63 tag)
- If QR is not readable by some wallets, verify required fields (like Tag 00, 01, 52, 53, 54, etc.)